CMake Project and Engine Management

Applies to FemboyEngine2D 0.2.0-alpha.1 and CMake 3.28 or newer.

Toolchain

The engine requires C++26:

- Windows: clang-cl 19+ with the MSVC STL, or MSVC 19.40+.
- Linux: Clang 19+ or GCC 15+.
- macOS/iOS: Apple Clang with C++26 support.
- Android: NDK r27+ with a C++26-capable Clang.

Ninja is the recommended generator. Visual Studio generators are supported but slower for this tree.

Configure a standalone build

```
cmake -S . -B build -G Ninja \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DCMAKE_C_COMPILER=clang-cl \
  -DCMAKE_CXX_COMPILER=clang-cl
cmake --build build --parallel 8
ctest --test-dir build --output-on-failure
```

Use `RelWithDebInfo` for tests. A Debug build is valid but several GPU and video tests become an order of magnitude slower and can hit CI timeouts.

Engine options

Option	Default	Meaning
`FBY_BUILD_EDITOR`	`ON`	Dear ImGui editor library and application.
`FBY_BUILD_SANDBOX`	`ON`	Sandbox executable.
`FBY_BUILD_SERVER`	`ON`	Dedicated server executable.
`FBY_BUILD_LOBBY`	`ON`	Lobby service executable.
`FBY_BUILD_TESTS`	`OFF`	Build the CTest suite.
`FBY_BUILD_TOOLS`	`ON`	`fby_fempak`, `fby_aotc`, `fby_fontc`, `fby_scene_tool`.
`FBY_BUILD_PLATFORM_PROVIDERS`	`ON`	Steam/EGS/GOG provider shared libraries.

`FBY_RHI_BACKEND`	`auto`	`auto`, `vulkan`, `metal`, or `null`.
`FBY_WITH_ASAN`, `FBY_WITH_UBSAN`	`OFF`	Sanitizer instrumentation for the engine and tests.
`FBY_ASAN_LEAKS`	`ON`	LeakSanitizer at exit on Linux and macOS.
`FBY_WITH_SCRIPT_VM`	`ON`	FemboyScript VM for editor and development builds.
`FBY_WITH_RNNOISE`	`ON`	RNNoise neural denoise tier.
`FBY_WITH_DLSS`, `FBY_WITH_FIDELITYFX`, `FBY_WITH_METALFX`	`OFF`	Vendor upscaler backends.
`FBY_DLSS_ROOT`	`third_party/dlss`	Local NVIDIA DLSS SDK root; not tracked.
`FBY_FIDELITYFX_ROOT`	`third_party/fidelityfx/Kits/FidelityFX`	Local AMD FidelityFX SDK root; not tracked.
`FBY_REQUIRE_VENDOR_SDKS`	`OFF`	Missing vendor SDK becomes a configure error instead of a warning.
`FBY_WITH_FSR1`, `FBY_WITH_FSR2`, `FBY_WITH_FRAME_GENERATION`	`OFF`	Compatibility flags; the features are core and runtime-gated.
`FBY_CONSOLE_APPS`	`OFF`	Keep a console window for the editor and games.

The engine works without a Vulkan SDK: `FBY_RHI_BACKEND=null` and a null RHI build are supported for headless CI. If Vulkan headers are unavailable, the editor still builds against the minimal Khronos headers vendored by SDL.

Public CMake aliases

Link only PascalCase aliases. They are the supported build-tree API.

Alias	Purpose
`FemboyEngine2D::Engine`	Runtime library. Link this from a game.
`FemboyEngine2D::Editor`	Editor library and UI stack.
`FemboyEngine2D::AssetCook`	BC7/ASTC texture cooker.
`FemboyEngine2D::AssetLib`	Pack and localization compiler.
`FemboyEngine2D::ShaderCompilerCore`	In-process GLSL/SPIR-V compiler.

Internal targets (`fby_engine`, `fby_editor`, `fby_fempak`, `fby_aotc`) and executable names are implementation details. They are not renamed for style and must not be used as a public contract. There are no `FemboyEngine::*` or `FemboyEngine2D::Runtime` compatibility aliases.

Consume the engine with add_subdirectory

```
cmake_minimum_required(VERSION 3.28)
project(MyGame LANGUAGES CXX)

set(FBY_ENGINE_ROOT "${CMAKE_CURRENT_SOURCE_DIR}/../.." CACHE PATH
"FemboyEngine2D root")
set(FBY_BUILD_EDITOR OFF CACHE BOOL "" FORCE)
set(FBY_BUILD_SANDBOX OFF CACHE BOOL "" FORCE)
set(FBY_BUILD_TESTS OFF CACHE BOOL "" FORCE)
set(FBY_BUILD_TOOLS OFF CACHE BOOL "" FORCE)
set(FBY_BUILD_PLATFORM_PROVIDERS OFF CACHE BOOL "" FORCE)

add_subdirectory("${FBY_ENGINE_ROOT}" "${CMAKE_BINARY_DIR}/_fby_engine"
EXCLUDE_FROM_ALL)

add_executable(MyGame GameMain.cpp)
target_link_libraries(MyGame PRIVATE FemboyEngine2D::Engine)
target_compile_definitions(MyGame PRIVATE
    FBY_PROJECT_ASSETS="${CMAKE_CURRENT_SOURCE_DIR}/assets")
```

The binary directory must be unique. Do not add the engine as the same binary directory as the game; CMake will reject the target names.

`FBY_PROJECT_ASSETS` is a compile definition, not an engine cache variable. The generated `GameMain.cpp` uses it to mount the project asset directory.

Generated Clean Slate project

`CreateProject()` writes:

```
MyGame/
  CMakeLists.txt
  project.femproj
  GameMain.cpp
  assets/
```

The generated CMakeLists disables editor, sandbox, tests, tools and storefront providers. It links only `FemboyEngine2D::Engine` and defines `FBY_PROJECT_ASSETS`. Open it with:

```
cmake -S MyGame -B MyGame/build -G Ninja -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake --build MyGame/build --parallel 8
```

The engine root must be passed by the project because the default project

location is `<engine>/Projects/<name>`. The generated CMake also accepts the `FBY_ENGINE_ROOT` environment variable; this keeps the project portable when it is moved to another machine or operating system:

```
export FBY_ENGINE_ROOT=/path/to/FemboyEngine2D
cmake -S MyGame -B MyGame/build
```

No absolute engine path is stored in `project.femproj`. Asset paths inside the project are project-relative; the generated CMake computes the absolute `FBY_PROJECT_ASSETS` path at configure time, so moving the project requires a reconfigure/rebuild, not a data migration.

An explicit cache value still wins:

```
cmake -S MyGame -B MyGame/build -DFBY_ENGINE_ROOT=/path/to/FemboyEngine2D
```

Build products

Target	Output
`fby_editor_app`	`build/bin/fby_editor`
`fby_sandbox`	`build/bin/fby_sandbox`
`fby_fempak`	`build/bin/fby_fempak`
`fby_aotc`	`build/bin/fby_aotc`
`fby_fontc`	`build/bin/fby_fontc`
`fby_scene_tool`	`build/bin/fby_scene_tool`

Tests and style gate

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=RelWithDebInfo
-DFBY_BUILD_TESTS=ON
cmake --build build --parallel 8
ctest --test-dir build --output-on-failure
python Source/Tools/Dev/CheckStyle.py
```

The CTest suite builds standalone executables with no test framework. They run headless and can be launched individually from `build/bin`.

RHI selection

```
cmake -S . -B build -DFBY_RHI_BACKEND=vulkan
cmake -S . -B build -DFBY_RHI_BACKEND=null
```

`null` is a real backend with deterministic frame accounting, not a stub. It is used by CI and by the generated project when no GPU is present. `auto` chooses Metal on Apple and Vulkan on Windows, Linux and Android when headers are available.

Adding a target

1. Add the source files to `Source/Engine/CMakeLists.txt` or the owning subsystem CMakeLists.
2. Link `FemboyEngine2D::Engine`, not `fby_engine`.
3. Add public include directories as `$<BUILD_INTERFACE:...>`.
4. Add a test through `fby_add_engine_test()` in `Source/Tests/CMakeLists.txt`.
5. Run the style gate and CTest.

Troubleshooting

- `cmake/EmbedShaders.cmake` not found: the engine was included from a game and `CMAKE_SOURCE_DIR` points at the game. Use the shipped `add_subdirectory` pattern; the engine derives its own root from `CMAKE_CURRENT_SOURCE_DIR`.
- `vulkan/vulkan.h` missing: configure with `-DFBY_RHI_BACKEND=null`, or install Vulkan-Headers. The editor has an SDL Khronos-header fallback.
- Legacy alias errors: replace `FemboyEngine::*` and `FemboyEngine2D::Runtime` with `FemboyEngine2D::Engine`.
- Tests time out: use `RelWithDebInfo`, not `Debug`.
- C++26 errors: update the compiler. C++23 is not supported.